



Async JavaScript – Recap Notes



Why Async JavaScript?

JavaScript is **single-threaded** and executes **code synchronously** by default. But **network requests**, **file reading**, or **timers** need time — if JS waited for them, it would block everything else.

So we use **asynchronous programming** to:

- **Avoid blocking** the main thread
 - Improve **performance and responsiveness**
 - Run **slow operations in the background**
-



Synchronous vs Asynchronous

♦ Synchronous Example:

```
console.log("Start");  
console.log("Processing...");  
console.log("End");
```

♦ Asynchronous Example:

```
console.log("Start");  
setTimeout(() => {  
  console.log("Inside Timeout");  
}, 2000);  
console.log("End");
```

Output:

```
Start  
End  
Inside Timeout
```

3 Core Ways to Handle Async in JS

1. Callbacks

2. Promises

3. Async/Await

1 Callbacks

A **function passed as an argument** to another function and executed later.

Example:

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data loaded");  
  }, 1000);  
}
```

```
fetchData(function(result) {  
  console.log(result);  
});
```

Problem: Callback Hell

Nested callbacks → hard to read, debug, and maintain

```
getUser(function(user) {  
  getPosts(user.id, function(posts) {  
    getComments(posts[0].id, function(comments) {  
      // Too nested!  
    });  
  });  
});
```

Promises

A **Promise** is an object representing the **eventual result** of an async operation.

```
const promise = new Promise((resolve, reject) => {  
  let success = true;  
  if (success) resolve("Success!");  
  else reject("Error!");  
});
```

```
promise.then(data => console.log(data))  
  .catch(err => console.error(err));
```

States of a Promise:

- Pending
- Fulfilled (Resolved)
- Rejected

Example: Simulated API

```
function fetchData() {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve("User data");  
    }, 2000);  
  });  
}
```

```
fetchData().then(data => console.log(data));
```

3 Async / Await

Introduced in ES2017, **async/await** is **syntactic sugar** over promises.

- ♦ **async** keyword: Makes a function return a Promise

- ♦ **await** keyword: Waits for the Promise to resolve

```
async function loadUser() {  
  const data = await fetchData(); // pause until resolved  
  console.log(data);  
}
```

✓ Full Example:

```
function fetchData() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve("Data loaded"), 1500);  
  });  
}
```

```
async function display() {  
  console.log("Start");  
  const result = await fetchData();  
  console.log(result);  
  console.log("End");  
}
```

```
display();
```

Real-life Async Use Cases

- API Calls (`fetch`, `axios`)
 - Event Listeners
 - Timers (`setTimeout`, `setInterval`)
 - File and database operations (in Node.js)
-

`fetch()` API – Making API Calls

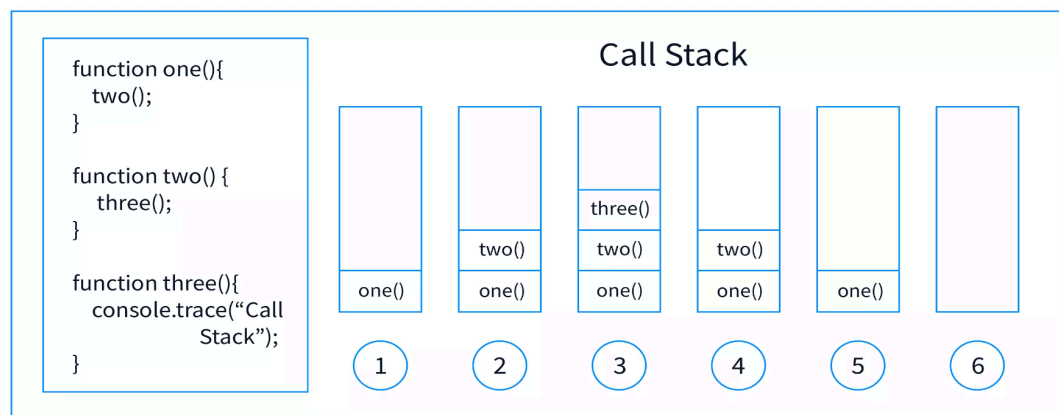
```
async function getUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Error fetching users", error);
  }
}
```

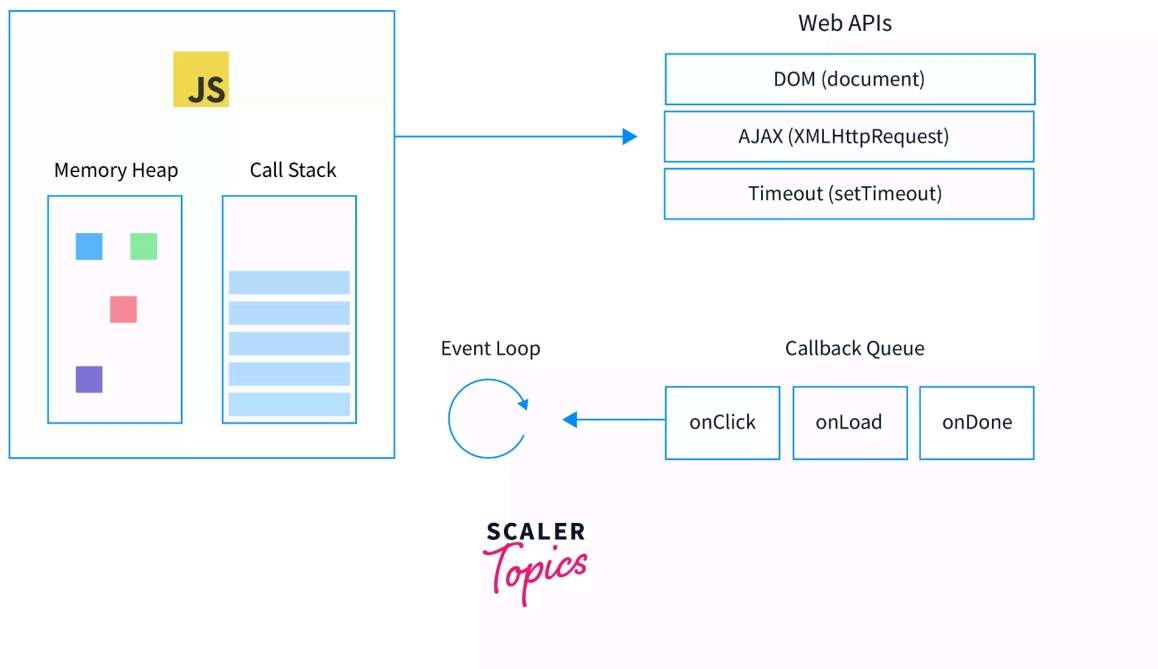
Common Mistakes

Mistake	Fix
Using <code>await</code> outside an <code>async</code> function	Wrap in an <code>async</code> function
Not handling errors	Use <code>try/catch</code> with <code>async/await</code> or <code>.catch()</code> with Promises
Mixing too many callbacks with promises	Stick to one style

⚙️ Event Loop & Async

- **Call Stack:** Executes current sync code
- **Web APIs:** Browser APIs like `setTimeout`, `fetch`
- **Callback Queue:** Stores async tasks (from Web APIs)
- **Event Loop:** Pushes async callbacks into the stack when it's empty





```
console.log("1");
```

```
setTimeout(() => {  
  console.log("2");  
}, 0);
```

```
console.log("3");
```

Output:

1
3
2

Summary

Concept	Description
Callback	Function passed into another function
Promise	Object that handles async result
Async/Await	Cleaner syntax for promises
Event Loop	Handles async execution in JS
fetch	Built-in function for making HTTP requests

Best Practices

- Always handle **errors** in async code
- Use **async/await** for cleaner flow
- Avoid deep **callback nesting**
- Prefer **modular code** with reusable async functions